Lab session 8

Objectives

- Build an application based on the patterns: hexagonal, domain model, and repository using a JPA mapper
- Evolve the layering architecture of the previous release version with JDBC to integrate the domain model and the mapper

Exercises

Get the initial skeleton

Download from the virtual campus the file ***ri.cws_base.jpa.2026.p8.skeleton.zip***. Uncompress it into a folder. It contains a maven root project with subprojects. Create a new workspace. If you use Eclipse import it using the option *Maven/Existing maven projects*.

This skeleton adds all the actors UI, the service interfaces definition and the acceptance tests.

Evolve the architecture from the previous version with JDBC

Follow this order:

- Upload to a new Eclipse workspace the maven projects (import as maven projects).
- Copy your domain model classes developed in previous practice to the `uo.ri.cws.domain package` of the *cws_services_jpa* project.
- Startup the attached HSQLDB database with the CarWorkshop app data.
- Look at the distribution of projects. There is one with the interfaces of the service layer, several presentation applications (one for each actor in the system; they are already complete) and the project for the services.
- Notice the interfaces of the service layer, they are the same as those of the previous JDBC version. Observe the use of the service, façade, factory, DTO and command patterns.
- Redo the classes that implement the *Transaction Scripts* (those in the *uo.ri.cws.application.service.*.commands* package). They now must be developed using model classes and repositories.

We will start with the CRUD cases of mechanics and the one for creating invoices.

Centralize transaction control

- Observe the `Command` and `CommandExecutor interfaces`. All the transaction scripts must implement the *Command* interface .
- Modify the classes that implement the facades so that each TS is executed by the `CommandExecutor` (one line of code for each method).

- Adds the utility class `Jpa.java` to the persistence layer. Look at their content and make sure you know what each of the public methods is for.
- Analyze the use of *Jpa* utility class in the `JpaCommandExecutor`.
- Adjust the *Factories* class in the `uo.ri.conf` package to get an instance of the `CommandExecutor`.
- Implement each implementation of the *Command*. Remember these no longer contain the domain logic (it is in the classes of the domain model); Its mission is to validate input parameters, invoke repositories, and delegate to domain classes to execute logic.

Complete repositories

- Look at the `package uo.ri.cws.application.repository`, it contains all the interfaces of the repositories. One for each entity. You'll see that they all inherit from the base interface *Repository*. Repositories use the metaphor of the collection (*add* and *remove*, but not update), they are neither TDG nor DAO.
- The `package uo.ri.cws.infrastructure.persistence.jpa` contains the implementations of those interfaces. There is a base implementation for common methods. You just must fill in the specific queries for each repository. Remember to retrieve the mapper with the utility class *Jpa*.
- Add the necessary queries in repository deployments.
    * Each method performs a query expressed in *JPQL* that must be in the `orm.xml file`.
    * At least add the *MechanicJpaRepository*, *InvoiceJpaRepository,* and *WorkOrderJpaRepository* with the necessary methods.

Complete queries

Repositories always return entities. Thay might come from JPQL queries. But, if we need derived reporting information (count interventions of a Mechanic, next invoice number, a report of Vouchers, etc.; the return type is not an entity) then we use Queries. In the end they also execute JPQL but do not need the typical add, remove, etc., operations of a Repository.

- Look at the `package uo.ri.cws.application.query`, it contains all the interfaces of the queries. The `package uo.ri.cws.infrastructure.persistence.jpa.queries` contains the implementations of those interfaces. You just must fill in the specific queries.

Complete the needed functionality until all the acceptance tests pass green.

About Carworkshop Extended (Delivery project)

Once you have all the previous tests running green then you can continue developing the delivery version. Download from the virtual campus the *ri.**cws_extended**.jpa.2026.**p8**.skeleton.zip*. It has the extended version of the database, the extended services interfaces, the new UI implementations and the corresponding acceptance tests.

Import the new skeleton into *a new workspace* and then copy the domain model classes, and all the implementations you have done (commands, repositories, queries, etc.) from the *cws_base* P8 project into the same package of the new project.

Now you must implement the use cases required by your UO code % 3 (see the wording). Use the proper *RunServiceTests_<n>* suite of acceptance tests to verify your implementation.